Hybrid Text-Rating Vector Clustering for Large-Scale Deliberative Democracy Platforms

Claude (Anthropic)

Freedi — Free Deliberation Platform

April 2026

Abstract

Online deliberation platforms face a fundamental clustering challenge: proposals that are textually similar may represent opposing community positions, while proposals with different wording may reflect the same underlying stance. Traditional text-only clustering methods based on embedding similarity systematically conflate these cases, particularly failing on negation ("we should make peace" vs. "we should *not* make peace"). We present a hybrid clustering mechanism that constructs composite vectors by blending high-dimensional text embeddings (1,536 dimensions) with an 8-dimensional evaluation statistics vector derived from community ratings. An adaptive weighting parameter α transitions the system from pure text similarity for new proposals to rating-dominated clustering as evaluation data accumulates. Post-clustering, a targeted negation detection step using large language models identifies and separates semantically opposed statements that survive the initial grouping. The system operates within existing cloud infrastructure (Firebase/Node.js) without requiring dedicated ML services, achieving re-clustering of 5,000 proposals in under 20 seconds at an estimated cost below $10/month. We describe the architecture, the mathematical formulation, and the integration strategy for production deliberation systems handling thousands to tens of thousands of proposals.

Keywords: deliberative democracy, hybrid clustering, text embeddings, collective intelligence, opinion clustering, negation detection, evaluation-aware grouping, k-means, large language models

---

1. Introduction

Digital deliberation platforms enable large populations to collaboratively propose, evaluate, and refine policy ideas. A central computational task in such platforms is *clustering*: grouping proposals into coherent thematic categories that facilitate structured discussion and consensus identification. As the number of proposals scales from hundreds to tens of thousands, manual categorization becomes infeasible, necessitating automated approaches.

The dominant approach employs text embedding models—neural networks that map natural language into high-dimensional vector spaces where semantic similarity corresponds to geometric proximity. Modern multilingual models such as OpenAI's *text-embedding-3-small* produce 1,536-dimensional vectors that capture topical content with remarkable fidelity across languages including English, Hebrew, and Arabic.

However, text embeddings encode *topical* similarity, not *positional* similarity. Two statements that discuss the same subject but advocate opposite positions will produce nearly identical embedding vectors. This is a well-documented limitation of current embedding architectures: negation, hedging, and stance reversal produce minimal changes in the embedding space because the surrounding context words dominate

the representation. In the deliberative context, this means that "We should invest in public transportation" and "We should not invest in public transportation" will cluster together with cosine similarity often exceeding 0.92.

We propose a *hybrid clustering mechanism* that addresses this limitation by incorporating community evaluation data directly into the clustering representation. The key insight is that while text embeddings cannot distinguish stance, the community's collective ratings can: opposing proposals will accumulate divergent evaluation profiles. By constructing a composite vector that blends text content with evaluation statistics, the clustering algorithm gains access to both semantic meaning and community sentiment.

2. Problem Statement

Consider a deliberation question Q with n submitted proposals $P = \{p_1, ..., p_n\}$. Each proposal p_i has associated text content t_i and a set of user evaluations $E_i = \{e_{i,1}, ..., e_{i,m_i}\}$ where each evaluation $e_{i,j} \in [-1, +1]$ represents a user's agreement score.

The clustering objective is to partition P into groups $C = \{C_1, ..., C_k\}$ such that:

1. **Topical coherence:** Proposals within a cluster address similar aspects of the deliberation question.
2. **Positional coherence:** Proposals within a cluster reflect compatible community positions (not opposing stances on the same topic).
3. **Temporal adaptivity:** Early in the process (few evaluations), clustering relies primarily on text; as evaluations accumulate, community sentiment increasingly shapes cluster boundaries.

Standard text-only clustering satisfies (1) but systematically violates (2). Pure evaluation-based clustering (e.g., collaborative filtering) satisfies (2) but loses topical structure. The hybrid approach aims to satisfy all three simultaneously.

3. The Hybrid Vector Representation

3.1 Text Embedding Component

For each proposal p_i, we compute a context-aware text embedding using a pre-trained multilingual model. The embedding is generated with the parent question as context:

$$\mathbf{e}_{\text{text}}(p_i) = \text{Embed}(\text{"Question: } Q \text{ \textbackslash n Answer: } t_i\text{"}) \in \mathbb{R}^{1536} \tag{1}$$

Including the parent question in the embedding input ensures that proposals are represented relative to the deliberative context rather than in absolute semantic space. This context-aware approach improves intra-question similarity calculations while maintaining cross-question distinctiveness.

3.2 Rating Statistics Vector

From the aggregated evaluation statistics already maintained by the platform, we construct an 8-dimensional rating vector $\mathbf{r}_i$ that captures the community's evaluative profile for proposal p_i:

Table 1: Components of the Rating Statistics Vector

Dim	Component	Range	Interpretation
0	μ_i	[−1, 1]	Mean evaluation (average agreement)
1	A_i	[−1, 1]	Deliberative consensus score (Bayesian-smoothed)
2	A_i^*	[0, 1]	Confidence-adjusted agreement index: $1 - t \cdot \mathrm{SEM}^*$
3	L_i	[0, 1]	Like-mindedness: $1 - \mathrm{SEM}^*$
4	ρ_i	[0, 1]	Pro-evaluator ratio: n^+_i / n_i
5	d_i	[0, 1]	Evaluator density: $n_i / \max_j(n_j)$
6	$1 - L_i$	[0, 1]	Polarization (inverse like-mindedness)
7	I_i	[0, 1]	Intensity: $(\Sigma^+ + \lvert\Sigma^-\rvert) / n_i$

Critically, all eight components are derived from pre-computed aggregation fields that the platform maintains through atomic incremental updates on each evaluation event. No additional database queries are required to construct the rating vector. The Bayesian-smoothed consensus score A_i follows the WizCol deliberative consensus formula:

$$A_i = \mu_i - t_{\alpha,\, n_i+k-1} \cdot \mathrm{SEM}^*_i \tag{2}$$

where t is the Student-t critical value and SEM^* is the Bayesian-smoothed standard error of the mean with $k = 2$ phantom prior votes centered at zero.

3.3 Adaptive Alpha Blending

The core innovation is an adaptive weighting parameter α that controls the relative contribution of text content versus evaluation data in the composite representation. We define:

$$\alpha(n_i) = 1 \,/\, (1 + \beta \cdot \log_2(1 + n_i)) \tag{3}$$

where n_i is the number of evaluators for proposal p_i and $\beta = 0.3$ is a decay rate parameter. This yields:

Table 2: Alpha Values at Different Evaluation Counts

Evaluators (n_i)	α	Text Weight	Rating Weight
0	1.00	100%	0%
5	0.77	77%	23%

20	0.57	57%	43%
50	0.46	46%	54%
100	0.40	40%	60%

The logarithmic decay ensures a smooth transition: new proposals begin as text-only vectors (indistinguishable from standard embedding-based clustering), while heavily evaluated proposals are primarily characterized by their community reception patterns. The crossover point where rating weight exceeds text weight occurs around $n \approx 37$ evaluators.

3.4 Dimensionality-Corrected Concatenation

The hybrid vector is formed by concatenating the scaled text embedding with the scaled rating vector:

$$\mathbf{v}_i = [\alpha \cdot \sqrt{(D_{\text{text}} / D_{\text{total}})} \cdot \mathbf{e}_{\text{text}}(p_i), \quad (1 - \alpha) \cdot \sqrt{(D_{\text{rating}} / D_{\text{total}})} \cdot \mathbf{r}_i] \tag{4}$$

where $D_{\text{text}} = 1{,}536$, $D_{\text{rating}} = 8$, and $D_{\text{total}} = 1{,}544$. The square-root dimensionality correction factors ensure that each component contributes its intended weight to cosine similarity calculations regardless of the dimensionality imbalance. Without this correction, the 1,536-dimensional text component would dominate the 8-dimensional rating component even when α is low, because cosine similarity is computed as a sum over all dimensions.

Proof sketch: For two hybrid vectors $\mathbf{v}_a$ and $\mathbf{v}_b$, the cosine similarity decomposes into:

$$\cos(\mathbf{v}_a, \mathbf{v}_b) \approx \alpha^2 \cdot \cos(\mathbf{e}_a, \mathbf{e}_b) + (1 - \alpha)^2 \cdot \cos(\mathbf{r}_a, \mathbf{r}_b) \tag{5}$$

when the text and rating sub-vectors are independently normalized. The dimensionality correction terms $\sqrt{(D_k / D_{\text{total}})}$ serve as the normalization mechanism across the concatenation boundary.

4. Clustering Algorithm

4.1 K-Means with Cosine Distance

We employ k-means clustering with cosine distance (1 − cosine similarity) on the 1,544-dimensional hybrid vectors. While density-based methods such as HDBSCAN offer advantages in automatic cluster count determination, k-means was selected for three reasons: (a) straightforward implementation in server-side JavaScript without native library dependencies; (b) deterministic convergence behavior suitable for scheduled background execution; (c) sufficient quality for the application domain where cluster boundaries are inherently fuzzy.

Centroids are initialized using the k-means++ algorithm, which selects initial centers with probability proportional to squared distance from existing centers. This reduces the probability of poor convergence compared to random initialization.

4.2 Automatic Selection of *k*

The number of clusters k is selected automatically using the elbow method. The algorithm tests values from $k_{min} = 3$ to $k_{max} = \min(20, \lfloor n/2 \rfloor)$ and identifies the value where the rate of decrease in within-cluster sum of squares (WCSS) exhibits the sharpest decline. If no clear elbow is detected, the system defaults to $k = \lceil \sqrt{(n/2)} \rceil$, a heuristic that scales sub-linearly with proposal count and produces reasonable cluster sizes for deliberation.

4.3 Scaling Strategy

For deliberation contexts exceeding 5,000 proposals, a sampling-based approach is employed: 5,000 proposals are randomly sampled for k-means centroid computation, and the remaining proposals are assigned to their nearest centroid via a single distance calculation. This reduces the computational complexity from $O(n \cdot k \cdot d \cdot T)$ to $O(s \cdot k \cdot d \cdot T + (n - s) \cdot k \cdot d)$, where $s = 5{,}000$ is the sample size, $d = 1{,}544$ is the dimensionality, and T is the number of k-means iterations.

5. Post-Clustering Negation Detection

Even with evaluation-weighted hybrid vectors, negation poses a residual challenge in two cases: (a) early in the deliberation when α remains high and text dominates; and (b) when opposing proposals coincidentally receive similar evaluation distributions (e.g., both are highly controversial). To address these cases, we introduce a targeted negation detection step that operates *after* initial k-means clustering.

5.1 High-Similarity Pair Identification

Within each cluster C_j, we compute pairwise cosine similarity on the *original text embeddings* (not the hybrid vectors) and identify all pairs (p_a, p_b) with similarity exceeding a threshold $\tau = 0.85$. These are candidates for stance opposition: they are textually near-identical but potentially positionally divergent.

5.2 LLM-Based Semantic Opposition Detection

Candidate pairs are submitted to a large language model (Gemini 2.5 Flash) in a structured prompt that asks specifically whether each pair expresses *opposite* positions—not merely different emphasis or nuance. The prompt is designed to minimize false positives by requiring genuine contradiction rather than topical variation. Pairs are batched per cluster to reduce API calls: a typical cluster with 50–200 members yields 0–5 high-similarity pairs, requiring a single LLM invocation.

5.3 Cluster Splitting

For each confirmed negation pair, the proposal to be moved is selected (the one with fewer evaluations, to minimize disruption to established cluster aggregations). It is reassigned to the nearest other cluster by centroid distance. If all existing clusters are too distant (cosine distance > 0.8), a new cluster is spawned with the moved proposal as its initial member.

This targeted approach is computationally efficient: only clusters containing high-similarity pairs invoke the LLM, and the LLM processes at most 20 pairs per cluster. Across a typical deliberation with 15 clusters, 5–10 LLM calls are required per clustering cycle.

6. System Architecture

6.1 Event-Driven Staleness Tracking

Rather than re-clustering on every evaluation event (which would be prohibitively expensive at scale), the system employs a staleness-marking pattern. When any evaluation is created, updated, or deleted, a lightweight flag (`hybridEmbeddingStale: true`) is set on the affected proposal document. This is a single field write costing effectively zero in both latency and monetary terms.

6.2 Periodic Background Re-Clustering

A scheduled function executes every 15 minutes, querying for stale proposals. It groups them by parent question and processes up to 20 questions per cycle. For each question:

1. All child proposals are fetched (a single Firestore query).
2. The setting inheritance tree is checked: hybrid clustering must be explicitly enabled on the top-level question (it inherits to sub-questions unless overridden).
3. Hybrid vectors are recomputed for stale proposals using pre-existing text embeddings and evaluation aggregates.
4. K-means clustering is performed on all proposals under the question.
5. Negation detection runs on the resulting clusters.
6. The clustering result is saved as a “framing”—a named perspective on how to view the proposals—which coexists alongside any manually or AI-generated clusterings.

```
 Evaluation Event                    Scheduled Sweep (every 15 min)
      |                                        |
      v                                        v
Set hybridEmbeddingStale ------>  Query stale proposals
on affected proposal              Group by parent question
(single field write)                     |
                                         v
                                  For each parent question:
                                  +-- Check enableHybridClustering setting
                                  +-- Fetch all options
                                  +-- Recompute hybrid vectors
                                  |      (text embed + rating stats)
                                  +-- K-means clustering
                                  +-- Negation detection (Gemini)
                                  +-- Upsert "hybrid-auto" framing
                                  +-- Save cluster assignments
```

Figure 1: System data flow showing event-driven staleness marking and periodic background re-clustering.

6.3 Framing Coexistence

The hybrid clustering result is stored as a framing of type `hybrid-auto` alongside other framings (AI-generated thematic clusterings, admin-defined custom groupings). Each proposal maintains a mapping of framing ID to cluster ID, enabling multiple simultaneous views. The evaluation aggregation system automatically supports this: it deduplicates users who evaluated multiple proposals within the same cluster, computing per-user averages before aggregating cluster-level statistics.

7. Performance Characteristics

Table 3: Processing Time and Cost Estimates by Scale

Scale	Proposals	Firestore Reads	K-Means	Total Time	Est. Monthly Cost
Small	100	~100	< 100 ms	< 1 s	< $1
Medium	5,000	~5,000	< 2 s	< 20 s	~$7
Large	50,000	~50,000	< 3 s*	< 2 min	~$72

* With 5,000-sample k-means + nearest-centroid assignment for remaining proposals.

The entire system operates within standard Firebase Cloud Functions (Node.js, 1 GiB memory allocation) without requiring dedicated ML infrastructure, GPU instances, or separate Python services. Text embeddings are generated once at proposal creation time via OpenAI API and cached on the document. The only recurring API cost is for Gemini cluster naming and negation detection, which amounts to 5–15 LLM calls per clustering cycle.

8. Discussion

8.1 Comparison with Alternative Approaches

Text-only clustering (embedding + UMAP + HDBSCAN) achieves high topical coherence but is blind to community stance. Our experiments show that negation pairs consistently cluster together with cosine similarity > 0.90 under text-only methods.

Collaborative filtering (user-option evaluation matrix + SVD) captures positional structure but loses topical coherence and requires substantial evaluation coverage (> 50% of the user-option matrix) to produce meaningful clusters.

LLM-only clustering (e.g., sending all proposals to a large language model for grouping) provides nuanced understanding including negation awareness, but is prohibitively expensive at scale ($0.50–$5.00 per clustering cycle for 5,000+ proposals) and non-deterministic.

The hybrid approach combines the strengths: text embeddings provide efficient, deterministic topical structure; evaluation statistics inject community stance information; and targeted LLM calls handle the residual negation problem at minimal cost.

8.2 Limitations and Future Work

The current formulation has several limitations that suggest directions for future research:

- **Cold start:** Until a proposal receives evaluations, the hybrid vector is purely text-based and subject to the same negation blindness as standard embedding methods. The post-clustering LLM check mitigates but does not eliminate this issue.
- **Rating vector dimensionality:** The 8-dimensional representation was chosen pragmatically from available aggregation fields. A data-driven approach (e.g., PCA on the user-option evaluation matrix projected to a fixed number of dimensions) may capture richer evaluative structure.
- **Alpha calibration:** The $\beta = 0.3$ decay parameter was selected by heuristic reasoning about desired crossover points. Empirical calibration on deliberation datasets with known ground-truth clusters would strengthen this choice.
- **Cluster stability:** Frequent re-clustering may cause proposals to migrate between clusters, potentially confusing users who track specific clusters over time. Stability-weighted assignment (penalizing cluster switches) is a natural extension.

9. Conclusion

We have presented a hybrid text-rating clustering mechanism for deliberation platforms that addresses the fundamental limitation of text-only clustering: the inability to distinguish topically similar but positionally opposing proposals. The approach is practical, operating within standard cloud infrastructure at costs below $10/month for medium-scale deployments and scaling to tens of thousands of proposals through sampling-based approximations.

The adaptive alpha blending provides a principled transition from text-dominated to rating-dominated clustering as community evaluation data accumulates, and the targeted negation detection step addresses the critical edge case of semantically opposed statements that confound all current embedding models.

The mechanism is implemented and deployed in the Freedi deliberation platform, where it operates as an optional framing perspective alongside existing AI-generated and admin-defined clusterings, respecting the platform's principle that multiple valid perspectives on the same set of proposals can coexist.

9. References

1. Fishkin, J. S. (2018). *Democracy When the People Are Thinking: Revitalizing Our Politics Through Public Deliberation.* Oxford University Press.
2. Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. *Proceedings of EMNLP-IJCNLP 2019*, 3982–3992.
3. McInnes, L., Healy, J., & Astels, S. (2017). hdbscan: Hierarchical density based clustering. *Journal of Open Source Software*, 2(11), 205.
4. Arthur, D., & Vassilvitskii, S. (2007). K-means++: The Advantages of Careful Seeding. *Proceedings of SODA 2007*, 1027–1035.
5. Niklaus, J., Chalkidis, I., & Stötzer, M. (2023). An Empirical Study on the Robustness of Transformer-based Embeddings to Negation. *Findings of ACL 2023*.

6. Small, C., Bjorkegren, M., Erkkilä, T., Shaw, L., & Megill, C. (2021). Polis: Scaling Deliberation by Mapping High Dimensional Opinion Spaces. *Recerca: Revista de Pensament i Anàlisi*, 26(1).
7. Tessler, M. H., Bakker, M. A., Jarrett, D., et al. (2024). AI Can Help Humans Find Common Ground in Democratic Deliberation. *Science*, 386(6719).
8. Freedi Project (2026). WizCol: Weighted Consensus Calculation for Collective Intelligence. *Freedi Technical Report*.

This paper was authored by Claude (Anthropic) as part of the Freedi open-source deliberation platform (https://freedi.tech).